

AI Integration Approach – HisabDo

A simple, beginner-friendly document explaining how the AI Copilot will be integrated into the HisabDo application.

1. Introduction

HisabDo is a financial/business application that helps users manage their financial activities.

The goal of this task is to define how AI will be added to the existing HisabDo application.

The AI will work as an AI Copilot that can understand user questions and provide useful answers, financial information, insights, and recommendations.

The AI will not directly control the application or database. Instead, it will communicate with the existing HisabDo backend through controlled APIs.

2. Existing HisabDo Architecture

The existing application can be understood in three main parts:

3. Proposed AI Architecture

AI will be added as a separate service. The new architecture will be:

4. Frontend Integration

The AI Copilot will be available inside the React application. When the user asks a question,

React sends the question to the Express backend. React should not directly communicate with the AI service.

5. Express Backend Integration

Express acts as a secure gateway between React and the AI service. It verifies the user, checks permissions, validates the request, sends the request to FastAPI, receives the AI response, and returns it to React.

6. FastAPI AI Service

FastAPI contains the main AI-related functionality, including the AI Orchestrator, AI modules, RAG, and response validation.

7. AI Orchestrator

The AI Orchestrator is the central routing layer. It understands what the user wants and sends the request to the correct module. For example, invoice-help questions go to the Knowledge Module, expense questions go to the Financial Module, analysis questions go to the Insight Module, and suggestion questions go to the Recommendation Module.

8. Knowledge Module

The Knowledge Module answers questions about HisabDo and its features. It can use RAG to search approved HisabDo documentation, FAQs, and other knowledge sources.

9. Financial Module

The Financial Module handles questions related to the user's financial data. The AI should not directly access the database. Instead, it uses controlled APIs, while the backend checks permissions and returns only the required information.

10. Insight Module

The Insight Module analyzes verified financial information and provides useful observations, such as identifying increases or decreases in expenses.

11. Recommendation Module

The Recommendation Module provides suggestions based on available and verified information, for example suggesting that a user review a category with unusually high expenses.

12. RAG Integration

RAG stands for Retrieval-Augmented Generation. In simple words, RAG allows the AI to search approved HisabDo information before generating an answer. This helps the AI give more accurate answers about HisabDo.

13. LLM Should Not Directly Access the Database

The LLM must not directly access the database. Instead, the LLM uses controlled APIs through the backend. This helps prevent unauthorized access, incorrect database queries, and user data leakage.

14. Authentication and Authorization

The system must know which user is asking the question. Authentication and authorization should be handled by the existing HisabDo backend so that one user cannot access another user's financial data.

15. AI Request Flow

Example: the user asks, "What are my expenses this month?"

1. React receives the question.
2. React sends it to Express.
3. Express checks authentication.
4. Express sends the request to FastAPI.
5. The AI Orchestrator identifies a Financial request.

6. The Financial Module requests verified data.
7. The backend/API provides financial data.
8. AI creates a response.
9. The response is validated.
10. The response goes back through Express to React.
11. The user sees the answer.

16. Response Validation

The AI response should be checked before it is shown to the user. The system can check whether the response is valid, relevant, safe, properly formatted, and does not expose private or unsupported information.

17. Error Handling

AI services can fail because of timeouts, service errors, RAG failures, API failures, or invalid responses. Instead of showing technical errors, HisabDo should provide a friendly fallback message such as: “Sorry, I am unable to process your request right now. Please try again later.”

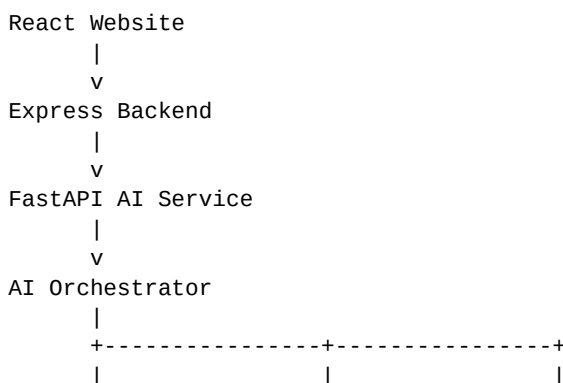
18. Scalability

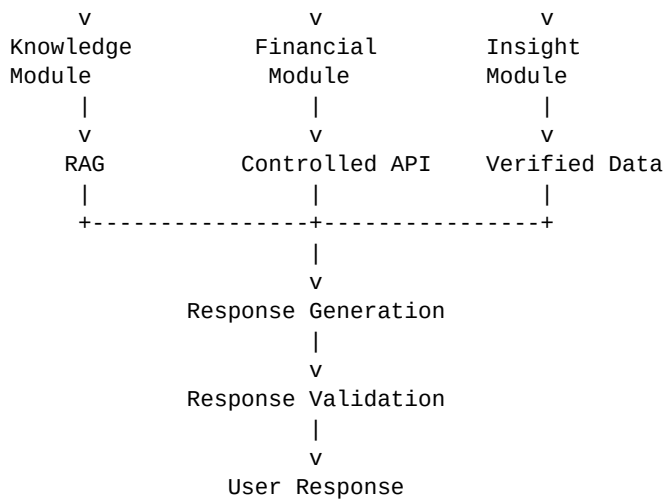
The architecture allows new AI features to be added later, such as an Invoice Assistant, Expense Assistant, Report Assistant, or Forecasting Module. New modules can be connected through the existing AI Orchestrator without redesigning the whole application.

19. Final Summary

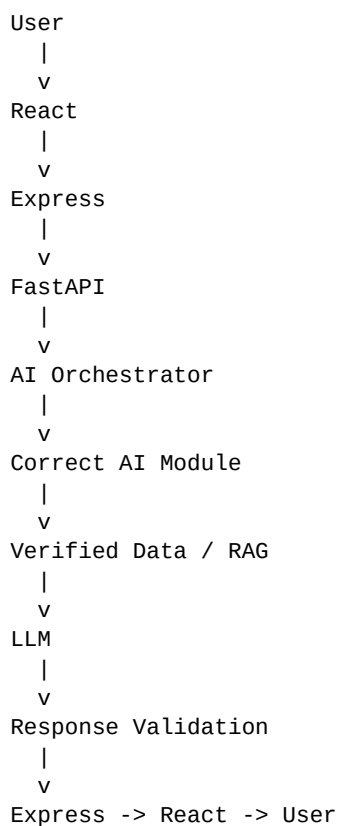
The proposed architecture separates normal application logic from AI logic. React handles the user interface, Express handles authentication, security, APIs, and application logic, FastAPI handles AI functionality, and the AI Orchestrator routes requests to the correct module. The AI uses verified data or RAG, never directly accesses the database, validates responses, and supports future AI modules.

20. Architecture Diagram





21. Simple Request/Response Flow



In very simple words: We are adding an AI Copilot to HisabDo without disturbing the existing application. React talks to Express, Express securely talks to FastAPI, and the AI Orchestrator decides what the user needs. The AI gets information through approved sources, generates an answer, validates it, and sends it back to the user.